

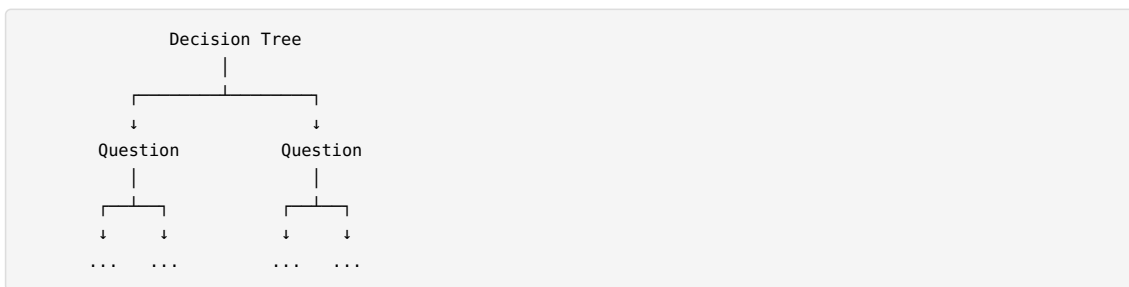
🌲 Lesson 8 — Random Forest & Ensemble Learning

You've completed:

1. ML Basics
2. Linear Regression
3. Regression Evaluation
4. Classification
5. Classification Evaluation
6. KNN
7. Decision Trees
8. Random Forest ← Today

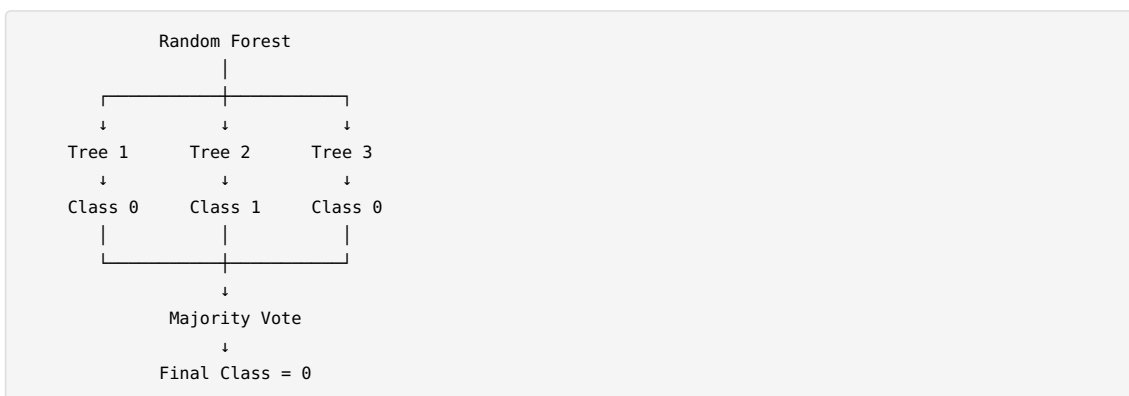
1. What is Random Forest?

Before Random Forest, remember the Decision Tree:



A Random Forest doesn't use just one tree.

It creates many Decision Trees and combines their predictions.



So:

Random Forest = Many Decision Trees + Combined Predictions

2. Why do we need many trees?

Suppose you ask one person whether a transaction is fraudulent.

They might make a mistake.

Now ask 100 people independently.

If most of them say:

```
Fraud
Fraud
Not Fraud
Fraud
Fraud
Fraud
Not Fraud
...
```

The majority answer is probably more reliable.

Random Forest works with a similar idea.

```
One Decision Tree
  ↓
Prediction
  ↓
Possible mistake
```

versus:

```
Many Decision Trees
  ↓
Multiple predictions
  ↓
Majority vote / averaging
  ↓
More robust prediction
```

This is called ensemble learning.

3. What is Ensemble Learning?

Ensemble means combining multiple models to produce a better prediction.

For example:

```
Model 1 ┌─┐
Model 2 ┌─┐
Model 3 ┌─┐ ──> Combined prediction
Model 4 ┌─┐
Model 5 ┌─┐
```

The basic idea is:

Several models working together can often perform better than one model alone.

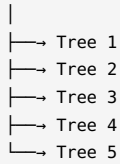
Random Forest is one of the most important examples of ensemble learning.

4. How does Random Forest work?

There are two important ideas:

Step 1 — Create many trees

```
Dataset
```



Step 2 — Each tree makes a prediction

For example:

Tree 1 → Fraud
Tree 2 → Fraud
Tree 3 → Not Fraud
Tree 4 → Fraud
Tree 5 → Fraud

Step 3 — Combine the predictions

Fraud = 4 votes
Not Fraud = 1 vote

Therefore:

Final Prediction → Fraud

5. Random Forest Classification

For classification, Random Forest generally uses majority voting.

Example:

Tree 1 → Class A
Tree 2 → Class B
Tree 3 → Class A
Tree 4 → Class A
Tree 5 → Class B

Votes:

Class A → 3
Class B → 2

Final:

Class A

6. Random Forest Regression

Random Forest can also perform regression.

Instead of voting for a class, the trees produce numerical predictions.

Example:

Tree 1 → ₹50 lakh
Tree 2 → ₹55 lakh
Tree 3 → ₹52 lakh

Tree 4 → ₹53 lakh
Tree 5 → ₹50 lakh

The forest combines them, commonly by averaging:

Final ≈ ₹52 lakh

Scikit-learn provides:

```
RandomForestClassifier()
```

for classification and:

```
RandomForestRegressor()
```

for regression.

7. Why is it called “Random” Forest?

This is very important.

Random Forest introduces randomness in the training process.

Two major sources are:

Random data samples

Different trees can be trained using different samples of the training data.



Random feature selection

When deciding a split, trees can consider a random subset of features.

For example, suppose you have:

CPU
Memory
Latency
Error Rate
Request Count
Network Traffic

One tree may consider:

CPU
Latency
Error Rate

while another may consider:

Memory
Request Count
Network Traffic

This helps make the trees different from one another.

8. Why not just use 100 identical trees?

If all trees were identical:

```
Tree 1 → Fraud
Tree 2 → Fraud
Tree 3 → Fraud
...
```

they would tend to make the same mistakes.

Random Forest tries to make the trees somewhat different.

Then their errors don't necessarily occur at the same places.

```
Tree 1 → mistake here
Tree 2 → mistake somewhere else
Tree 3 → correct
Tree 4 → correct
Tree 5 → correct
      ↓
Majority = Correct
```

That's one of the major strengths of Random Forest.

9. Decision Tree vs Random Forest

Decision Tree	Random Forest
One tree	Many trees
Can overfit easily	Usually more resistant to overfitting
Simple	More complex
Easy to visualize	Harder to visualize
Fast to train	Usually slower
Can have unstable predictions	Generally more stable
Single model	Ensemble

Think:

```
Decision Tree
[Tree]
```

versus:

```
Random Forest
[Tree] [Tree] [Tree] [Tree] [Tree]
  [Tree] [Tree] [Tree] [Tree]
[Tree] [Tree] [Tree] [Tree] [Tree]
```

10. Let's implement Random Forest

We'll use the same Breast Cancer dataset we've already used for classification.

Step 1 — Import

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

Notice this:

```
from sklearn.ensemble import RandomForestClassifier
```

ensemble contains ensemble-learning algorithms.

Step 2 — Load dataset

```
data = load_breast_cancer()

X = data.data
y = data.target
```

Remember:

```
X → Features
y → Target
```

Step 3 — Train/test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

We're using:

```
80% → training
20% → testing
```

Step 4 — Create Random Forest

```
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
```

The important parameter here is:

```
n_estimators=100
```

It means approximately:

Create 100 decision trees in the forest.

So:

```
RandomForestClassifier
|
├─ Tree 1
├─ Tree 2
```

```
|— Tree 3
|— ...
|— Tree 100
```

11. Train the model

```
model.fit(X_train, y_train)
```

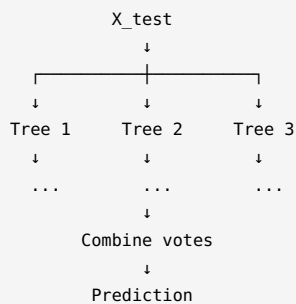
The Random Forest learns patterns from the training data.

12. Make predictions

```
predictions = model.predict(X_test)
```

Now the 100 trees participate in producing predictions.

Conceptually:



13. Calculate accuracy

```
accuracy = accuracy_score(y_test, predictions)

print("Accuracy:", accuracy)
```

14. Complete code ☆

Run this entire cell:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = load_breast_cancer()

# Features and target
X = data.data
y = data.target

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
```

```

        random_state=42
    )

    # Create Random Forest
    model = RandomForestClassifier(
        n_estimators=100,
        random_state=42
    )

    # Train
    model.fit(X_train, y_train)

    # Predict
    predictions = model.predict(X_test)

    # Evaluate
    accuracy = accuracy_score(y_test, predictions)

    print("Accuracy:", accuracy)

```

You should get a high accuracy, although the exact result can depend on the configuration/version.

15. Important Random Forest parameters ☆

There are several parameters you'll commonly see.

n_estimators

Number of trees.

```
RandomForestClassifier(n_estimators=100)
```

Example:

```

n_estimators=10
→ 10 trees

n_estimators=100
→ 100 trees

n_estimators=500
→ 500 trees

```

Generally, more trees can improve stability, but they also increase computation.

max_depth

Maximum depth of each tree.

```

RandomForestClassifier(
    n_estimators=100,
    max_depth=10
)

```

This helps control tree complexity.

Remember from Decision Trees:

```

Very deep tree
↓
Can memorize training data
↓

```

Overfitting

min_samples_split

Minimum number of samples required to split an internal node.

```
min_samples_split=5
```

min_samples_leaf

Minimum number of samples that must be present in a leaf.

```
min_samples_leaf=2
```

These parameters help control complexity.

16. Does Random Forest require scaling?

This is an important connection with KNN.

For KNN:

```
Distance matters
↓
Scaling important
```

For Decision Trees:

```
Thresholds/splits
↓
Scaling generally unnecessary
```

Random Forest is made from Decision Trees.

Therefore:

Random Forest generally does not require feature scaling.

For example, you normally don't need:

```
StandardScaler()
```

just to use Random Forest.

17. Random Forest vs KNN vs Decision Tree

Now you have learned three different ideas:

KNN

```
"Which training examples are closest to me?"
```

Decision Tree

```
"What questions can I ask to separate the data?"
```

Random Forest

"What do many different decision trees collectively predict?"

This is an important mental model.

18. Why Random Forest is powerful for real-world ML

Random Forest is popular because it can handle:

- numerical features
- many features
- nonlinear relationships
- classification
- regression
- complex feature interactions

For example, in SentinelAI you might have:

```
CPU usage
Memory usage
Latency
Error rate
Request count
Network traffic
...
```

A single Decision Tree might learn:

```
IF CPU > 90
AND latency > 500
THEN failure
```

Another tree might learn:

```
IF error_rate > 5
AND memory > 85
THEN failure
```

Another:

```
IF latency > 800
AND CPU > 70
THEN failure
```

The Random Forest combines many such patterns.

19. Random Forest and SentinelAI 🏠

This is where today's lesson becomes especially relevant to your project.

Your SentinelAI system has telemetry such as:

```
CPU
Memory
Latency
Error Rate
```

You ultimately want:

```
Telemetry
  ↓
Feature Engineering
  ↓
ML Model
  ↓
Failure Probability
  ↓
Severity
```

Random Forest can be one candidate for the ML model:

```
CPU ———|
Memory ———|
Latency ———|
Error Rate —|
          ↓
        Random Forest
          ↓
    Failure Probability
          ↓
    ┌───┴───┐
    ↓       ↓
Low Risk   High Risk
```

And Random Forest can provide feature importance, which can help answer:

“Which telemetry features contributed most to the model’s decisions?”

For example, you might eventually see something like:

```
Latency    → 0.42
Error Rate  → 0.31
CPU         → 0.18
Memory      → 0.09
```

That would give you useful insight into which signals are most influential.

20. One important concept: Feature Importance ☆☆☆

Random Forest can expose:

```
model.feature_importances_
```

Let’s try it.

```
importance = model.feature_importances_

print(importance)
```

You’ll get one importance value for each feature.

Because our dataset has many features, let’s display them nicely:

```
for name, value in zip(data.feature_names, model.feature_importances_):
    print(name, ":", value)
```

Conceptually:

Feature	Importance
-----	-----
mean radius	0.05
mean texture	0.02
mean perimeter	0.06
...	

The values indicate how much each feature contributed to the forest's splitting decisions, according to the model's built-in importance measure.

Important: feature importance is useful, but it is not the same thing as causal importance or a guaranteed explanation of an individual prediction.

Lesson 8 — Remember These

1.

Random Forest = collection of Decision Trees

2.

Classification → majority voting

3.

Regression → combine numerical predictions, commonly by averaging

4.

`n_estimators` → number of trees

5.

Random Forest usually doesn't require feature scaling

6.

Random Forest is an Ensemble Learning algorithm

7.

Randomness makes individual trees different

8.

Many trees can produce a more robust model than one tree

9.

`feature_importances_` can provide feature-level importance

Practice Questions

Try answering these without looking above:

Q1

What is the main difference between a Decision Tree and Random Forest?

Q2

If:

```
n_estimators=200
```

what does it mean?

Q3

Suppose five trees predict:

```
Tree 1 → 1  
Tree 2 → 0  
Tree 3 → 1  
Tree 4 → 1  
Tree 5 → 0
```

What is the final classification?

Q4

Does Random Forest generally require `StandardScaler`?

Q5

What is ensemble learning?

Q6

What does this give us?

```
model.feature_importances_
```

Next lesson

Lesson 9 — Naive Bayes

We'll learn:

```
Probability  
  ↓  
Bayes' Theorem  
  ↓  
Naive Bayes  
  ↓  
Classification  
  ↓  
Spam / Text / Document Classification
```